

Day 19 : Riverpod State Management

1. Introduction to Provider

Riverpod is a modern state management library for Flutter developed by the creator of Provider. It is used to manage and share application state efficiently across different screens and widgets. Riverpod helps developers build scalable, maintainable, and reactive applications by separating business logic from the user interface.

2. Why we use Riverpod?

- **Better State Management:** It simplifies managing and updating application state.
- **NoBuildContext Required:** Unlike Provider, Riverpod does not depend on BuildContext to access data.
- **Compile-Time Safety:** Many errors are detected during compilation, reducing runtime crashes.
- **Improved Testing:** Business logic can be tested easily without involving UI components.

3. Steps For the Riverpod Installation

Step 1: Install Packages

Open terminal and run: `flutter pub add flutter_riverpod`

Step 2: Setup ProviderScope

Riverpod requires the whole app to be wrapped inside ProviderScope.

Inside main.dart :

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'home_screen.dart';

void main() {
  runApp(
    ProviderScope(
      child: MyApp(),
    ),
  );
}

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      home: HomeScreen(),
    );
  }
}
```

Step 3: Learn Provider Types

Provider (Read Only)

Used for constant values.

```
final appNameProvider = Provider<String>((ref) {  
    return "Riverpod Demo";  
});
```

Use:

```
class HomeScreen extends ConsumerWidget {  
    @override  
    Widget build(BuildContext context, WidgetRef ref) {  
  
        final appName = ref.watch(appNameProvider);  
  
        return Scaffold(  
            appBar: AppBar(  
                title: Text(appName),  
            ),  
        );  
    }  
}
```

StateProvider (Simple State)

Used for simple states like counter.

```
final counterProvider = StateProvider<int>((ref) => 0);
```

Example :

```
class CounterScreen extends ConsumerWidget {  
  
    @override  
    Widget build(BuildContext context, WidgetRef ref) {  
  
        final count = ref.watch(counterProvider);  
  
        return Scaffold(  
            appBar: AppBar(  
                title: Text("Counter"),  
            ),  
            body: Center(  
                child: Text(  
                    "$count",  
                    style: TextStyle(fontSize: 30),  
                ),  
            ),  
            floatingActionButton: FloatingActionButton(  
                onPressed: () {  
                    ref.read(counterProvider.notifier).state++;  
                },  
                child: Icon(Icons.add),  
            ),  
        );  
    }  
}
```

StateNotifierProvider (Complex State)

Used when application logic becomes complex.

```
import 'package:flutter_riverpod/flutter_riverpod.dart';

class TodoNotifier extends StateNotifier<List<String>> {

  TodoNotifier() : super([]);

  void addTodo(String task) {
    state = [...state, task];
  }

  void removeTodo(String task) {
    state = state.where((item) => item != task).toList();
  }
}
```

Provider

```
final todoProvider =
  StateNotifierProvider<TodoNotifier, List<String>>((ref) {
    return TodoNotifier();
  });
```

Use :

```
final todos = ref.watch(todoProvider);

ref.read(todoProvider.notifier).addTodo("Study Riverpod");
```

FutureProvider (API Calls)

Best for fetching API data.

```
final newsProvider = FutureProvider<List<dynamic>>((ref) async {

  final response = await http.get(
    Uri.parse(
      'https://jsonplaceholder.typicode.com/posts',
    ),
  );

  return jsonDecode(response.body);
});
```

UI:

```
class NewsScreen extends ConsumerWidget {

  @override
  Widget build(BuildContext context, WidgetRef ref) {

    final news = ref.watch(newsProvider);

    return Scaffold(
      appBar: AppBar(title: Text("News")),

      body: news.when(
        data: (data) {
          return ListView.builder(
            itemCount: data.length,
            itemBuilder: (context, index) {
              return ListTile(
                title: Text(data[index]['title']),
              );
            },
          );
        },

        loading: () => Center(
          child: CircularProgressIndicator(),
        ),

        error: (error, stack) =>
          Center(child: Text(error.toString())),
      ),
    );
  }
}
```

StreamProvider (Streams)

Used for real-time data.

```
final timerProvider =
StreamProvider<int>((ref) async* {

  int i = 0;

  while (true) {
    await Future.delayed(Duration(seconds: 1));
    yield i++;
  }
});
```

Usage :

```
final timer = ref.watch(timerProvider);

timer.when(
  data: (value) => Text("$value"),
  loading: () => CircularProgressIndicator(),
  error: (e, s) => Text("Error"),
);
```

4. Important Riverpod Concepts

1. ConsumerWidget

```
class HomeScreen extends ConsumerWidget {
  @override
  Widget build(BuildContext context, WidgetRef ref) {

    return Scaffold();
  }
}
```

2. Consumer

```
Consumer(
  builder: (context, ref, child) {

    final count = ref.watch(counterProvider);

    return Text("$count");
  },
)
```

3. ref.watch()

```
final count = ref.watch(counterProvider);
```

4. ref.read()

```
ref.read(counterProvider.notifier).state++;
```

5. ref.listen()

ref.listen() is used to listen for changes in a provider without rebuilding the widget. It is useful for showing SnackBars, dialogs, or navigating to another screen when the state changes.

```
ref.listen(counterProvider, (previous, next) {
  print("Counter changed: $next");
});
```

5. Auto Dispose Providers

Riverpod can automatically remove unused providers from memory.

```
final userProvider = FutureProvider.autoDispose((ref) async {  
  return fetchUser();  
});
```

6. Family Modifier

The .family modifier allows passing parameters to providers.

```
final userProvider =  
FutureProvider.family<User, int>((ref, id) async {  
  return fetchUser(id);  
});
```

7. Provider Lifecycle

Riverpod providers have their own lifecycle.

Important lifecycle methods:

- onDispose() → Called when provider is removed.
- onCancel() → Called when no widget listens.
- onResume() → Called when listening resumes.

```
final timerProvider = Provider((ref) {  
  ref.onDispose(() {  
    print("Provider disposed");  
  });  
});
```

8 . Hooks Riverpod

Hooks Riverpod combines Flutter Hooks with Riverpod to reduce boilerplate code.

```
class HomeScreen extends HookConsumerWidget {  
  
  @override  
  Widget build(BuildContext context, WidgetRef ref) {  
  
    final controller = useTextEditingController();  
  
    return Scaffold(  
      body: TextField(  
        controller: controller,  
      ),  
    );  
  }  
}
```

OUTPUT :

10:37

News



Henri Veesaar Missed Out on \$5M+ in Potential NIL Amid UNC Star's NBA Draft Slide, CBB Coach Says – Bleacher Report

Former North Carolina standout Henri Veesaar fell all the way to the No. 52 pick of the 2026 NBA draft and likely co...

Bleacher Report



Top aide to former NYC mayor Eric Adams indicted on bribery charges

10:37

News



Strong earthquake hits Venezuela as buildings collapse in Caracas – follow live – BBC

The magnitude 7.2 and 7.5 quakes caused panic in the capital as residents evacuated.

BBC News



Henri Veesaar Missed Out on \$5M+ in Potential NIL Amid UNC Star's NBA Draft Slide, CBB Coach Says –